



# Moving the Backend from Supabase to a Locally Hosted Server

Impact analysis, required services, trade-offs, and a recommendation

Prefects Discipline · Discipline Management Dashboard

15 August 2026

SUMMARY

This report examines what it would take to replace the Supabase backend (Postgres + REST API + auth) with infrastructure you run yourself, the new services you would have to operate, the trade-offs involved, and whether it is worth the effort for this application.

## 1. Current architecture

The app is a Next.js (App Router) client that talks directly to Supabase through the supabase-js SDK. There is no server-side API of its own — all data access happens in the browser.

| Aspect             | Today                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Database           | Supabase Postgres (10 tables: users, students, strikes, blackmarks, goldmarks, punishments, comments, lists, list_attendance_sessions, list_attendance_records) |
| Data access        | ~85 supabase-js query call sites across 12 files (pages, components, lib modules)                                                                               |
| Auth               | Custom: browser fetches the user row, verifies the bcrypt hash client-side, stores the user object in localStorage. Supabase Auth is NOT used.                  |
| Authorization      | Frontend-only role checks (user.role in localStorage); no RLS, no server-side enforcement                                                                       |
| Realtime / storage | None used                                                                                                                                                       |
| Configuration      | Supabase URL + anon key hardcoded in lib/supabase.ts                                                                                                            |

## 2. Frontend changes required

### 2.1 Data access layer

- Replace the Supabase client in lib/supabase.ts with a thin API client (a fetch wrapper) pointing at your local server. If you deploy PostgREST (recommended, see §3), postgrest-js can keep the query syntax nearly identical, which minimizes churn.
- Rewire the ~85 call sites across 12 files from .from("table").select().insert().update().delete() chains to REST calls — app/page.tsx, app/discipline/page.tsx, app/student/[admissionNo]/page.tsx, app/users/page.tsx, app/lists/\*\*, components/DashboardDailyStats.tsx, lib/students.ts, lib/listAttendance.ts, lib/disciplineReport.ts, lib/AuthContext.tsx, components/ConfirmPasswordModal.tsx.
- Adjust error handling: supabase-js returns { data, error }; a REST API returns HTTP status codes. The ~15 alert/error paths need small rewrites.

### 2.2 Authentication

- Move password verification to the server. Login becomes POST /auth/login { username, password }; the server verifies the bcrypt hash and returns a session token (JWT or httpOnly cookie).
- Store the token (plus the user object for UI role gating) instead of storing the user row directly; send the token with every request.
- Existing bcrypt hashes in users.password carry over unchanged — no password resets are needed for your users.
- The user-management screens (app/users/page.tsx) currently read/write users.password in the browser; the password column should stop being exposed to the client and be handled server-side only.

## 2.3 Security posture

- Role checks are frontend-only today (a forged localStorage user object grants admin UI). Since you are doing the work anyway, enforce roles server-side (Row Level Security on Postgres, or middleware in the API) — otherwise the local backend inherits today's weakness.
- Move the hardcoded URL/anon key out of lib/supabase.ts into an env var (e.g. NEXT\_PUBLIC\_API\_URL).

## 2.4 What does NOT need to change

- No realtime subscriptions and no file storage exist — nothing to port there.
- Pages and UI logic (discipline records, lists, attendance, reports, dashboards) stay as-is; only their data-fetching plumbing changes.

## 3. New services you would need to run

| Service             | Purpose                   | Notes / options                                                                                                                                                                                                                                                                                 |
|---------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL          | The database itself       | Postgres 15/16. Restore the existing schema + data (pg_dump from Supabase).                                                                                                                                                                                                                     |
| API layer           | Expose tables to the app  | Recommended: PostgREST — mirrors Supabase's table API, so most queries keep working with minimal edits. Alternative: custom Node/Express/Fastify API with Prisma/Drizzle — full control, more code to maintain.                                                                                 |
| Auth service        | Login + session tokens    | bcrypt verify server-side, issue JWTs or httpOnly cookies. With PostgREST: JWT secret + RLS policies that check the token role.                                                                                                                                                                 |
| Reverse proxy + TLS | HTTPS in front of the API | Caddy (auto Let's Encrypt) or nginx. Required if the app is hosted remotely; strongly recommended on LAN too.                                                                                                                                                                                   |
| Process manager     | Keep services alive       | systemd unit or pm2: start on boot, restart on crash.                                                                                                                                                                                                                                           |
| Backups             | Protect the data          | Scheduled pg_dump (e.g. daily cron) + off-site copy. Supabase does this for you today.                                                                                                                                                                                                          |
| Monitoring          | Know when it breaks       | At this scale: uptime check, disk-space cron, journald logs are enough.                                                                                                                                                                                                                         |
| Networking          | Make the server reachable | The Next.js app must reach the API from the browser. If the frontend stays on Vercel, the local server needs a public endpoint — which largely cancels the privacy benefit and adds attack surface. True local hosting means self-hosting the Next.js app too (next start) on the same network. |

### WATCH OUT

Biggest hidden cost: operations. You now own patching, uptime, TLS, backups, and security for every one of those services. At this app's scale (a handful of tables, a few users, ~2,700 students) the running cost is small, but the responsibility is yours.

## 4. High-level migration path

- **Step 1** — Export the Supabase database (pg\_dump or the Supabase dashboard backup) and restore it into local Postgres.
- **Step 2** — Deploy the API layer (PostgREST or custom) + auth endpoint behind the reverse proxy, with the same table/column names.
- **Step 3** — Swap the client: new API client + token handling in lib/AuthContext.tsx and the ~85 call sites.
- **Step 4** — Enforce roles server-side (RLS or middleware) before cutover.
- **Step 5** — Test each module: login, dashboard, records, student detail, users, lists, attendance, PDF reports.
- **Step 6** — Cut over (env var or DNS). Keep Supabase read-only as a rollback path until the local stack has been stable for a week.

## 5. Pros and cons

| Pros                                                                                        | Cons & p                                                                                                                                     |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Full data ownership — student records stay on your hardware, no third party involved.       | You own operations: patching, uptime, backups, disk space, and security for the whole stack.                                                 |
| No dependence on Supabase availability, pricing changes, or vendor roadmap.                 | Security is now on you — TLS, firewall, auth tokens, RLS. A misconfiguration exposes student data.                                           |
| Predictable cost: hardware, not per-project SaaS pricing that grows with usage.             | You lose Supabase Studio (used today to browse/manage data and users) and managed extras (realtime, storage, edge functions) if ever needed. |
| Lower latency on a school LAN.                                                              | One-time effort: ~85 call sites, auth rework, deployment, data migration — plus ongoing maintenance (OS, Postgres, PostgREST updates).       |
| No request rate limits or fair-use caps.                                                    | Single point of failure unless you add redundancy.                                                                                           |
| You can add real server-side authorization (RLS), fixing today's frontend-only role checks. | If the Next.js app stays hosted remotely, the local DB must be exposed publicly — cancelling the privacy win.                                |
| Works even if the internet connection drops (fully on-prem).                                | Future maintainers need Postgres/Linux ops knowledge.                                                                                        |

## 6. Effort estimate (one developer, part-time)

| Work item                                 | Estimate          |
|-------------------------------------------|-------------------|
| Data export & import                      | 0.5 day           |
| API layer + client swap (PostgREST path)  | 2–3 days          |
| Server-side auth + tokens                 | 1–2 days          |
| Server-side role enforcement (RLS)        | 0.5–1 day         |
| Deployment: VM/LAN, TLS, systemd, backups | 1–2 days          |
| Testing every module + cutover            | 1–2 days          |
| Total                                     | "H 1–1.5 weeks    |
| Ongoing operations                        | "H 1–2 hours/week |

## 7. Is it worth it?

For this application — a small, self-contained discipline tracker with 10 tables, a handful of staff users, and the free tier of Supabase — moving to a self-hosted backend is not worth it unless one of these is true:

- School policy or data-protection rules require student records to stay on premises (data residency).
- You want zero external dependency — no rate limits, no reliance on a third party's uptime, or the app must work with no internet.
- Cost certainty matters and you expect Supabase's usage-based pricing to grow meaningfully.
- You (or your team) are comfortable owning Linux/Postgres operations and security long-term.

If none of those apply today, staying on Supabase is the lower-risk, lower-effort choice: it is already running, free at this scale, and its managed Postgres, backups, and TLS are worth more than they cost you.

### If you do move

- Use PostgREST on local Postgres rather than a custom API — it keeps the frontend diff small and the schema portable.

- Self-host the Next.js app on the same network; don't expose the local DB to the public internet.
- Add RLS / server-side role enforcement while you're in there — it fixes the current frontend-only authorization weakness.
- Set up daily `pg_dump` backups with an off-site copy on day one, not later.

### **Cheapest middle path**

Stay on Supabase but schedule a periodic local export (Supabase 'Download backup' or a nightly script that pulls the tables). That gives you an escape hatch — a full, restorable copy of the data — so the move stays cheap and reversible if a concrete driver appears later.

#### **RECOMMENDATION**

Verdict: for a school discipline tracker of this size, the trade-off leans 'stay on Supabase' unless privacy/compliance or independence is an explicit requirement. The migration is very doable (~1–1.5 weeks), but it trades a managed service for a permanent operations job. Revisit only when a concrete driver exists.